



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Ingeniería Del Software II

Sección 1

Ejercicio 2: La Falacia de los Recursos y el Cono de Incertidumbre Contexto

Profesor:

Márquez Félix

Grupo: Ingenio y Código

Aguilera Rhixeidys V-30.851.503

Bravo Yonkeiner V-30.994.057

Carreño Brayan V-32.015.527

Garcia Nicole V-30.809.865

Ciudad Guayana, septiembre de 2026

Introducción

La estimación y planificación en la ingeniería de software se caracterizan por una alta incertidumbre y complejidad inherente. Históricamente, la gestión tradicional ha intentado aplicar modelos deterministas a la construcción de código, asumiendo erróneamente que las tareas de desarrollo son perfectamente divisibles e independientes. En este contexto, uno de los errores gerenciales más comunes ante un proyecto que presenta retrasos es la suposición de que la relación entre tiempo, esfuerzo y recursos humanos es lineal. El presente informe analiza conceptual y matemáticamente el escenario crítico de un proyecto a dos meses de su fecha límite, evaluando el impacto de duplicar el equipo de desarrollo. A través del prisma de la Ley de Brooks y las métricas de flujo, se demostrará por qué inyectar más personal en etapas tardías resulta contraproducente y se propondrán alternativas de gestión viables, tanto predictivas como ágiles, para mitigar la crisis sin comprometer la integridad técnica del producto.

Ejercicio 2: La Falacia de los Recursos y el Cono de Incertidumbre

- **Contexto del problema:** Faltan 2 meses para la entrega. El proyecto está retrasado. Se ordena duplicar el equipo (de 5 a 10 programadores) asumiendo que esto recuperará el tiempo perdido para cumplir con el diagrama de Gantt.

1. Análisis Conceptual y Matemático de la Ecuación Tiempo = Esfuerzo / Recursos

La ecuación lineal Tiempo = Esfuerzo / Recursos es una falacia en el desarrollo de software porque trata a la construcción de código como un trabajo perfectamente particionable e independiente (como cavar una zanja). En la ingeniería de software, las tareas son altamente interdependientes y el trabajo es de naturaleza cognitiva.

Matemáticamente, la falacia se demuestra analizando los canales de comunicación. La cantidad de líneas de comunicación en un equipo se calcula mediante la fórmula $C = \frac{n(n-1)}{2}$, donde n es el número de miembros.

- Con **5 desarrolladores**, existen $\frac{5(4)}{2} = 10$ canales de comunicación.
- Con **10 desarrolladores**, existen $\frac{10(9)}{2} = 45$ canales de comunicación.

El esfuerzo gastado en coordinar, sincronizar y comunicar crece de forma geométrica, no lineal. Esto es la base matemática de la **Ley de Brooks**, la cual establece empíricamente que agregar recursos humanos a un proyecto de software retrasado lo retrasará aún más.

2. Factores de Sobrecarga al Duplicar el Equipo

Al duplicar la plantilla en la recta final del proyecto, el equipo sufrirá un impacto negativo inmediato debido a tres factores concurrentes:

- **Sobrecarga de Comunicación:** Al pasar de 10 a 45 canales de comunicación, el tiempo que los desarrolladores invierten en reuniones, alineación de arquitectura, resolución de dependencias y debates técnicos aumenta drásticamente, restando horas efectivas de escritura de código.
- **Onboarding (Curva de Aprendizaje):** Los 5 programadores nuevos no conocen el dominio del negocio, la arquitectura del sistema, ni los estándares de codificación. Para que sean productivos, los 5 programadores originales (los únicos que conocen el sistema) deberán detener su trabajo para capacitar, guiar y revisar el código de los novatos, generando una caída en la productividad del equipo original.
- **Complejidad de Integración:** Con 10 desarrolladores escribiendo código simultáneamente, habrá un mayor número de ramas (branches) paralelas. Esto incrementará exponencialmente los conflictos de fusión (merge conflicts), los errores de regresión y los "detallitos del código", lo que funcionará como una acumulación súbita de deuda técnica que friccionará la operatividad.

3. Alternativas de Gestión ante la Crisis de Cronograma

Para salvar el proyecto sin violar los principios de la ingeniería de software, se deben aplicar enfoques que manejen la realidad empírica y restrinjan las variables flexibles:

1. **Enfoque Predictivo Tradicional (Gestión de Alcance y Ruta Crítica):** En lugar de modificar los recursos (que impactan negativamente) o el tiempo (que está fijo en 2 meses), se debe negociar el **Alcance (Tamaño)**. Utilizando el método del Camino Crítico (CPM), el equipo debe identificar estrictamente las funcionalidades que son indispensables para la operación del sistema. Todas las tareas que no pertenezcan al camino crítico estructural o que representen funcionalidades "deseables" deben ser recortadas o diferidas a una versión 1.1.

2. **Enfoque Ágil/Híbrido (Métricas de Flujo y Triage):** El líder debe abandonar la falsa precisión del cronograma de Gantt inicial y utilizar **Métricas de Flujo**. Se debe medir el Throughput (tasa de entrega real) del equipo actual de 5 personas. Con estos datos históricos, se puede ejecutar una simulación de Monte Carlo para determinar qué porcentaje del Backlog tiene un 85% de probabilidad de ser completado en las 8 semanas restantes. Con este pronóstico probabilístico respaldado por datos reales, se realiza un "triage" con los stakeholders para priorizar únicamente el valor más alto que cabe dentro de esa capacidad demostrada, manteniendo explícitamente el tiempo para refactorización y evitar que la deuda técnica hunda la velocidad actual.

Conclusión

Duplicar el número de programadores en la fase final de un proyecto de software crítico no es una solución viable para acortar los tiempos de entrega, sino una falacia matemática que agrava el retraso. Como se ha demostrado, el aumento exponencial de los canales de comunicación, sumado al esfuerzo requerido para la capacitación de los nuevos integrantes (onboarding) y la mayor fricción en la integración del código, generan una sobrecarga que penaliza severamente la velocidad del equipo original. La ingeniería de software moderna establece que, ante una crisis de cronograma, las variables a ajustar deben ser el alcance o las expectativas, no los recursos humanos. Las organizaciones deben abandonar la ilusión del control absoluto y transicionar hacia modelos empíricos, apoyándose en la reducción del alcance mediante el Camino Crítico (CPM) o en la gestión de la incertidumbre a través del rendimiento histórico del equipo (Throughput), garantizando así entregas realistas y protegiendo la calidad arquitectónica del sistema.

Referencias Bibliográficas

- **Brooks, F. P. (1975).** The Mythical Man-Month: Essays on Software Engineering. Addison-Wesley.
- **McConnell, S. (2004).** Code Complete: A Practical Handbook of Software Construction (2nd ed.). Microsoft Press. (Capítulo 8: The Cone of Uncertainty).
- **Vacanti, D. (2019).** Actionable Agile Metrics for Predictability: An Introduction. InfoQ / Lean-Agile Press.